MMDU MULLANA OFFICIAL SYLLABUS • BCSE-008 • UNIT 2

Unit 2: Core Programming Concepts, Operators & Control Structures

Subject: Computational & Problem Solving Using C (BCSE-008) • **Exam Weightage:** 24 - 28 Marks • **Study Time:** 38 mins

C Tokens, 32 ANSI keywords, identifier grammar, constants, primitive data types table, 9-tier operator precedence hierarchy, implicit integer promotions vs explicit typecasting, header files taxonomy, switch fall-through, and loop lifecycles with nested 2D matrix traversal.

1. Tokens, Keywords, Identifiers & Primitive Data Types

1. C Tokens Architecture

Tokens are the atomic lexical units recognized by the C compiler. They comprise:

- **Keywords:** 32 reserved words in C89/C90 (e.g., `auto`, `volatile`, `sizeof`, `typedef`).
- **Identifiers:** User-defined symbolic names for variables and functions. Rules: Must begin with `[a-zA-Z_]`, followed by alphanumeric or underscore characters. Case-sensitive.
- **Constants:** Immutable literals (Integer literals `100`, Octal `012`, Hex `0x64`, Floating-point `3.14`, Character `'A'`, String `"Hello"`).

---

2. Primitive Data Types, Memory Storage & Specifiers

Data Type	Storage in RAM	Format Specifier	Range (32-bit Architecture)	Encoding Scheme
char	1 Byte (8 bits)	`%c`	-128 to 127	ASCII / 2's Complement
unsigned char	1 Byte	`%c`, `%u`	0 to 255	Pure binary magnitude
int	4 Bytes (32 bits)	`%d`, `%i`	-2,147,483,648 to 2,147,483,647	Signed 2's Complement
unsigned int	4 Bytes	`%u`	0 to 4,294,967,295	Pure binary magnitude
float	4 Bytes	`%f`	$\approx 3.4 \times 10^{-38}$ to 3.4×10^{38} (6 decimals)	IEEE 754 Single Precision
double	8 Bytes	`%lf`	$\approx 1.7 \times 10^{-308}$ to 1.7×10^{308} (15 decimals)	IEEE 754 Double Precision

| **void*** | 4 / 8 Bytes | `%p` | Valid Virtual Memory Addresses | Unsigned Hexadecimal |

2. Complete 9-Tier Operator Precedence & Associativity Hierarchy

Complete 9-Tier Operator Hierarchy Master Table

Tier	Category	Operators	Associativity	Key Exam Rules & Behaviors
1 (Highest)	Postfix / Subscript	`() [] -> . ++ --`	Left-to-Right	Function call `()`, array subscript `[]`, and member access `->` resolve first.
2	Unary	`+ - ! ~ ++ --` `(type)` `&` `sizeof`	**Right-to-Left**	Prefix operators, address-of `&`, dereference `*`, and explicit cast evaluate right-to-left.
3	Multiplicative	`* / %`	Left-to-Right	Modulus `%` requires integral operands only (compiler error on floats).
4	Additive	`+ -`	Left-to-Right	Standard arithmetic addition and subtraction.
5	Relational	`< <= > >=`	Left-to-Right	Yields Boolean 1 (True) or 0 (False).
6	Equality	`== !=`	Left-to-Right	Lower precedence than relational: `a < b == c` evaluates as `(a < b) == c`.
7	Logical	`&&` (AND) then `||`		(OR)	Left-to-Right	**Short-Circuit:** In `A && B`, if A=0, B is skipped! In `A`		B, if A=1, B is skipped!
8	Ternary Conditional	`? :`	**Right-to-Left**	`condition ? expr1 : expr2`. Nested ternaries

Tier	Category	Operators	Associativity	Key Exam Rules & Behaviors
				group right-to-left.

| **9 (Lowest)** | Assignment & Comma | `= += -= *= /= %=` | `,` | **R-to-L** (=) | L-to-R (,) | Comma operator evaluates left-to-right and returns rightmost expression. |

3. Type Conversion: Implicit Promotion vs Explicit Typecasting

1. Implicit Type Conversion (Usual Arithmetic Conversions)

The C compiler automatically promotes lower-ranked types to higher-ranked types during binary expression evaluation to preserve precision:

```
bool → char → short → int → unsigned int → long → unsigned long → float → double → long double
```

> ⚠️ **The Classic Integer Division Hazard:**

```
> int a = 5, b = 2;
> float result = a / b; // result evaluates to 2.000000, NOT 2.5!
>
```

> **Root Cause:** Both `a` and `b` are integers. The compiler performs integer division (`5 / 2 = 2`), discarding the remainder before promoting the integer `2` to `float 2.000000` .

---

2. Explicit Typecasting

The programmer explicitly overrides default conversions using the cast operator `(target_type)expression` :

```
int a = 5, b = 2;
float result = (float)a / b; // Promotes 'a' to float first → 5.0 / 2 yields 2.500000!
```

4. Standard Header Files Taxonomy

Directory of Key ANSI C Standard Library Headers

Header File	Primary Purpose & Declarations	Canonical Library Functions
	Standard Input/Output Streams & File Operations	`printf`, `scanf`, `fopen`, `fscanf`, `fprintf`, `fclose`, `fgets`
	Dynamic Memory, Process Control, Conversions	`malloc`, `calloc`, `realloc`, `free`, `exit`, `atoi`, `rand`
	Null-terminated string manipulation & memory blocks	`strlen`, `strcpy`, `strncpy`, `strcat`, `strcmp`, `memcpy`, `memset`
	Mathematical & trigonometric computational functions	`pow`, `sqrt`, `sin`, `cos`, `tan`, `floor`, `ceil`, `fabs`
	Character classification and case transformation	`isalpha`, `isdigit`, `isalnum`, `isspace`, `toupper`, `tolower`
	Standard Boolean definitions (C99 onwards)	`bool`, `true` (1), `false` (0)

| | Architecture platform limits for primitive integers | `INT_MAX`, `INT_MIN`, `CHAR_BIT`, `LONG_MAX` |

5. Control Structures: Decision-Making, Loops & Jump Statements

1. Decision-Making & Switch Fall-Through Mechanics

In C `switch(expression)`, the expression must evaluate to an integral or character constant. Omitting the `break;` statement causes **Fall-Through**, where control cascades sequentially into subsequent cases:

```
switch (grade) {
    case 'A': printf("Excellent "); // Omitting break causes fall-through!
    case 'B': printf("Good ");      // Executes if grade is 'A' OR 'B'!
              break;
    default:  printf("Average");
}
```

---

2. Looping Lifecycles & Jump Statements

Loop Construct	Control Type	Minimum Iterations	Best Suited For
`for` **Loop**	Entry-Controlled (Condition evaluated before body)	0 times	Deterministic iterations with known step bounds.
`while` **Loop**	Entry-Controlled (Condition evaluated before body)	0 times	Event-driven loops where termination depends on runtime state.
`do-while` **Loop**	**Exit-Controlled** (Condition evaluated after body)	**1 time**	Interactive menu loops requiring at least one display pass.

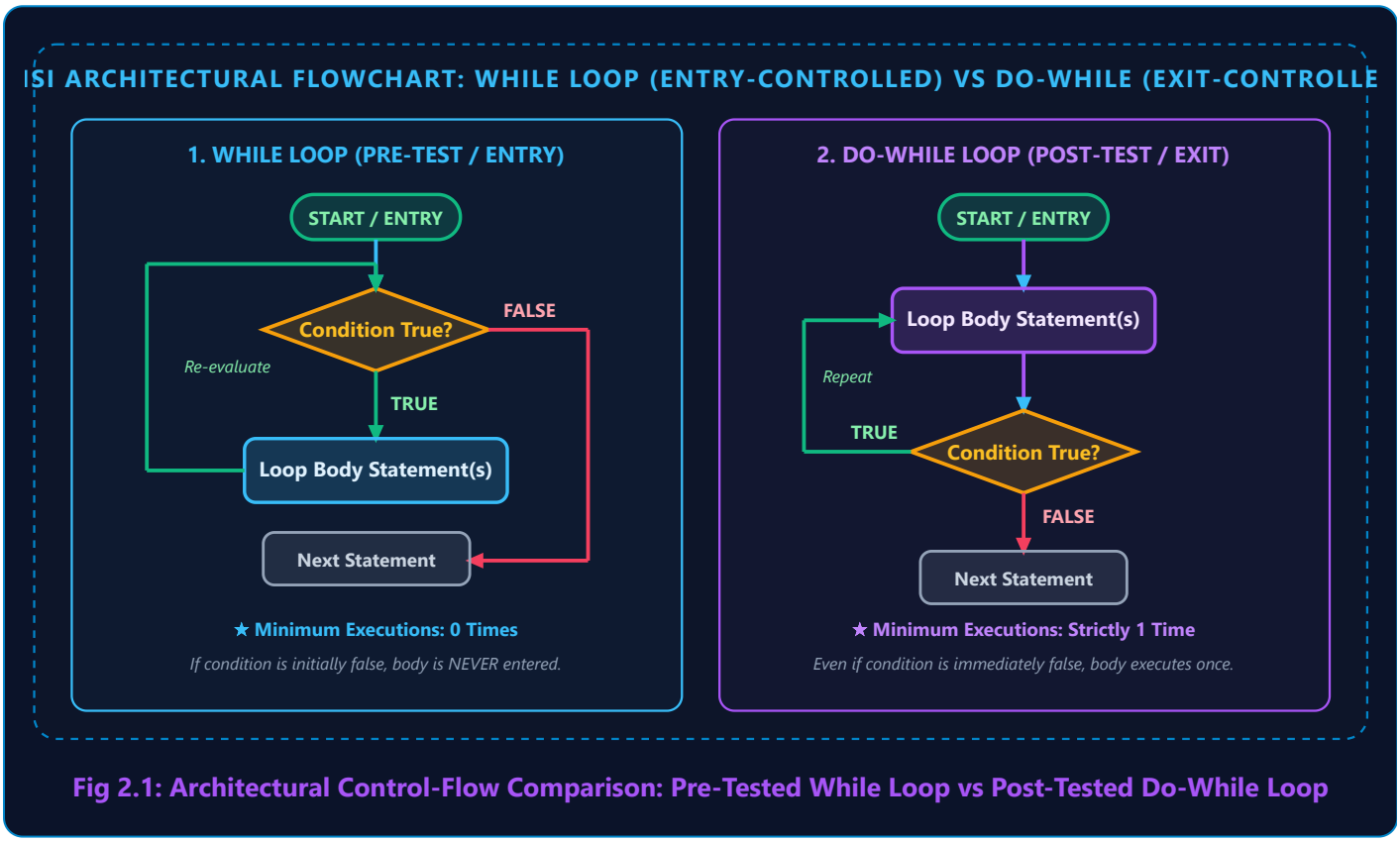


Fig 2.1: Architectural Control-Flow Comparison: Pre-Tested While Loop vs Post-Tested Do-While Loop

Jump Statements:

- `break;` : Instantly aborts the innermost enclosing loop or switch block.
- `continue;` : Skips the remaining statements of the current pass and jumps directly to the loop update expression (`i++`).
- `goto label;` : Unconditional jump directly to a labeled statement. Strongly discouraged in structured programming.

6. University Examination Solved Questions (Unit 2)

University Solved Examination Problems (MMDU Pattern)

Question 4 (1 Mark): Operator Precedence Evaluation

Question: Evaluate the expression order for: `int x = 2, y = 3; int z = x + y ++x;`

```
Binary Operator Tree with root '+' having left operand 'x' and right subtree 'y * (++x)'.
```

Step-by-Step Solution:

1. *Prefix increment* `++x` *(tier 2 unary) executes first:* `x` *becomes* `3`.

2. *Multiplicative operator* (tier 3) evaluates: `y x = 3 3 = 9` .

3. Additive operator `+` (tier 4) evaluates: `x + 9 = 3 + 9 = 12` .

Result: `z = 12` .

Question 14 (2 Marks): Short-Circuit Logic Evaluation

Question: What are the final values of `a` and `b` after executing:

```
int a = 0, b = 5;
if (a && ++b) { /* empty */ }
```

Step-by-Step Solution:

```
Left Operand (a == 0) is FALSE ➔ Short-Circuit Triggered ➔ Right expression (++b) is BYPASSED!
```

Under ANSI C rules for logical `&&`, if the left operand evaluates to `0` (false), the outcome is guaranteed false. The compiler bypasses evaluation of the right operand. Thus, `++b` is never executed, and `b` remains **5**.
Final Values: `a = 0, b = 5` .

Question 18 (4 Marks): Nested Loop Matrix Multiplication

Question: Explain the 3-tier nested loop structure required for multiplying matrices $A_{M\times K}$ and $B_{K\times N}$.

Step-by-Step Solution:

```
Outer Loop (i: 0 to M-1) ➔ Iterates across Rows of Matrix A
  Middle Loop (j: 0 to N-1) ➔ Iterates across Columns of Matrix B
    Inner Accumulator (k: 0 to K-1) ➔ Dot product: C[i][j] += A[i][k] * B[k][j]
```

```
void multiplyMatrices(int A[2][2], int B[2][2], int C[2][2]) {
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            C[i][j] = 0;
            for (int k = 0; k < 2; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

Complexity: Executes $M \times N \times K$ times, yielding time complexity $O(M \cdot N \cdot K)$ ($O(N^3)$ for square matrices).